

REDROB HACKATHON

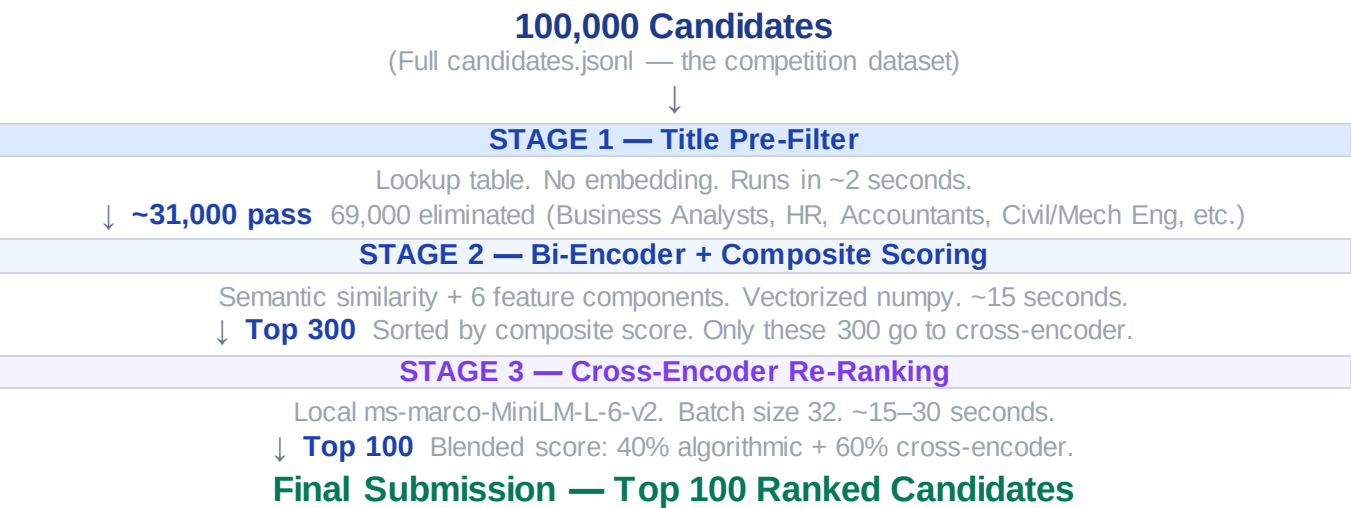
Pipeline Flow — Updated Section

Backend Logic Bible — Section 2 Replacement

Section 2 — Complete Pipeline Flow with Candidate Counts

This section replaces the previous pipeline table. The key architectural decision is the three-stage funnel: title filter eliminates most irrelevant candidates first, bi-encoder narrows to a precise shortlist, cross-encoder makes the final quality judgment. Each stage has a different speed/accuracy tradeoff.

2.1 The Three-Stage Funnel (Full 100K Dataset)



CRITICAL RULE: Cross-encoder NEVER runs on all candidates. It only ever sees the shortlisted top-300. This is intentional — cross-encoder takes ~100ms per pair. On 100K that would take 2.8 hours. On 300 it takes 30 seconds. The bi-encoder's job is to narrow the field fast so the cross-encoder can do deep, accurate judgment on a small set.

2.2 Each Stage — What It Does and Why

Stage	Name	Input Count	Output Count	How It Works	Time on CPU
1	Title Pre-Filter	100,000	~31,000	Lookup table maps each current <code>title</code> to a relevance score 0–1. Titles like 'Business Analyst' → 0.02 (eliminated). Titles like 'ML Engineer' → 1.0 (kept). No embedding, no model — just a dictionary lookup.	~2 seconds
2A	Bi-Encoder Semantic	~31,000	scored list	Pre-computed embeddings loaded from <code>.npy</code> file. Three JD query vectors × all candidate vectors via vectorized cosine similarity. Fast numpy matrix multiplication — no model inference at this step.	~5 seconds
2B	Feature Scoring	~31,000	scored list	6 components: skill trust, career quality, experience curve, location, education, behavioral multiplier. Composite score per candidate. Combined with semantic score from 2A.	~10 seconds

Stage	Name	Input Count	Output Count	How It Works	Time on CPU
2C	Sort and Shortlist	~31,000	Top 300	Sort all ~31,000 scored candidates descending. Take top 300. These are the candidates cross-encoder will evaluate.	~1 second
3	Cross-Encoder	300	Top 100	Build (JD query text, candidate summary text) pair per candidate. Run cross-encoder in batches of 32. Blend CE score with composite score. Sort by blended score. Take top 100.	~15–30 seconds

STAGES 2A + 2B + 2C are all ONE phase in the code: They run sequentially on the same ~31,000 candidates and together they are called the 'First-Pass Composite Scorer'. The breakdown above shows how they work internally. From outside the system it looks like: Title Filter → Score Everything → Shortlist 300 → Cross-Encoder.

2.3 How the Flow Scales for Different Input Sizes

The architecture automatically adapts based on how many candidates you load. The cross-encoder shortlist cap adjusts dynamically:

Input Size	After Title Filter	Bi-Encoder Shortlist	Cross-Encoder Input	Final Output	Total Runtime
100,000 (competition)	~31,000	Top 300	300 pairs	Top 100	~88 seconds
10,000	~3,100	Top 300	300 pairs	Top 100	~45 seconds
1,000	~310	Top 150	150 pairs	Top 100	~20 seconds
100 (demo sample)	Skip title filter — all 100 pass	Top 80	80 pairs	Top 50	~8 seconds
50 (official hackathon sample)	Skip title filter — all 50 pass	All 50	50 pairs	Top 20–50	~5 seconds

SKIP TITLE FILTER for small inputs: If input has fewer than 500 candidates, skip the title filter entirely. Pass all candidates directly to bi-encoder scoring. This is because with 50 candidates you want to rank all of them — eliminating any would reduce the output below 100. The title filter is a performance optimization for large datasets, not a quality gate.

2.4 Cross-Encoder Shortlist Cap Logic

The shortlist size sent to the cross-encoder is not always 300. It adapts:

Condition	Shortlist Size Sent to CE	Reason
Input > 1,000 candidates	300 (hard cap)	Large dataset — only top-300 from composite scoring go to CE. Budget: ~30s.

Condition	Shortlist Size Sent to CE	Reason
Input 200–1,000 candidates	$\min(300, \text{input} \times 0.30)$ — e.g. 500 candidates → 150	Medium dataset — 30% of input up to 300 max.
Input 50–200 candidates	All candidates	Small dataset — run CE on everything. 200 pairs takes ~20 seconds.
Input < 50 candidates	All candidates	Very small — CE on all. Negligible time.

2.5 Why Not Run Cross-Encoder on All 100K

Approach	Time for 100K	Feasible?	Why We Don't Do This
Cross-encoder on all 100K	~2.8 hours	NO	Way over the 5-minute constraint
Cross-encoder on all ~31K (post title filter)	~52 minutes	NO	Still way over budget
Cross-encoder on top 1,000	~100 seconds	BORDERLINE	Cuts too close to 5-minute limit. Leaves no budget for other stages.
Cross-encoder on top 300 (our approach)	~30 seconds	YES	Leaves ~60 seconds for all other stages. Comfortable within 5-minute budget.
Cross-encoder on top 100 only	~10 seconds	YES but risky	Too small — good candidates at ranks 101–200 from the algorithm are missed.

WHY 300 IS THE RIGHT NUMBER: 300 is the sweet spot between coverage and speed. The ~745 genuine ML/AI candidates in the 100K dataset will almost all appear in the top-300 after composite scoring. Going higher (500+) wastes CE budget on marginal candidates. Going lower (100) risks missing good candidates the algorithm slightly underrated.

2.6 Bi-Encoder vs Cross-Encoder — Why Both Are Needed

Property	Bi-Encoder	Cross-Encoder
How it works	Encodes JD and candidate SEPARATELY into vectors. Scores = cosine similarity between vectors.	Reads JD and candidate TOGETHER in one forward pass. Score = relevance of the pair.
Speed	Very fast — vectorized matrix multiply across all candidates at once	Slow — one forward pass per (JD, candidate) pair
Quality	Good for retrieval — finds candidates that are semantically similar to the JD	Better for ranking — understands the interaction between JD requirements and candidate description
Scale	Can handle 100K in seconds	Can handle ~300 in 30 seconds within the time budget
Role in pipeline	Fast first-pass: narrows 31K to 300	Deep final judgment: accurately re-ranks the 300
Misses	Candidates whose description uses different words for the same concept	Catches what bi-encoder missed — reads the actual content deeply

Property	Bi-Encoder	Cross-Encoder
	(partially caught by multi-query approach)	

The combination is the standard neural information retrieval architecture used in production search systems at Google, LinkedIn, and Flipkart. Bi-encoder for recall, cross-encoder for precision. This is what the JD is describing when it asks for 'ranking systems and retrieval pipeline experience' — you built one.

Implementation Notes for the Pipeline

Where Each Stage Lives in Your Code

Stage	File	Key Function
Title Filter	config.py + feature_extractor.py	TITLE_RELEVANCE_MAP dict lookup. Returns 0.02 for non-tech, 1.0 for core ML/AI. Runs in a single list comprehension.
Bi-Encoder Similarity	precompute_embeddings.py (offline) + scoring_engine.py (online)	Offline: embed all candidates, save to .npy. Online: load .npy, matrix multiply with 3 JD query vectors, weighted sum.
Feature Scoring	feature_extractor.py	Calls skill_scorer, career_analyzer, behavioral_scorer per candidate. Returns feature dict.
Shortlist	scoring_engine.py	np.argsort(scores)[::-1][:300]. One line. Returns indices of top-300 candidates.
Cross-Encoder	cross_encoder_reranker.py	Build (query, passage) pairs for top-300. cross_encoder.predict(pairs, batch_size=32). Blend scores. Return final top-100.

The One Thing to Get Right

MOST COMMON MISTAKE: Running cross-encoder BEFORE shortlisting. Always shortlist with composite score first, THEN run cross-encoder on the shortlist. Never pass raw unsorted candidates to the cross-encoder — it will be slow and the quality gain from CE will be wasted on irrelevant candidates.

END — Pipeline Flow Section

Replace Section 2 in Backend Logic Bible with this document.